S1
Consider following Relation
Account (Acc_no, branch_name,balance)
Branch(branch_name,branch_city,assets)
Customer(cust_name,cust_street,cust_city)
Depositor(cust_name,acc_no)
Loan(loan_no,branch_name,amount)
Borrower(cust_name,loan_no)
Create above tables with appropriate constraints like primary key,
foreign key, not null etc.

```
CREATE TABLE Branch (
   branch_name VARCHAR(50) PRIMARY KEY,
   branch_city VARCHAR(50) NOT NULL,
   assets DECIMAL(12,2)
);

CREATE TABLE Account (
   acc_no INT PRIMARY KEY,
   branch_name VARCHAR(50),
   balance DECIMAL(12,2) CHECK (balance >= 0),
   FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);

CREATE TABLE Customer (
   cust_name VARCHAR(50) PRIMARY KEY,
   cust_street VARCHAR(100),
   cust_city VARCHAR(50)
);

CREATE TABLE Depositor (
   cust_name VARCHAR(50),
   acc_no INT,
   PRIMARY KEY (cust_name, acc_no),
   FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
   FOREIGN KEY (acc_no) REFERENCES Account(acc_no)
);

CREATE TABLE Loan (
   loan_no INT PRIMARY KEY,
   branch_name VARCHAR(50),
   amount DECIMAL(12,2) CHECK (amount > 0),
   FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);
```

```
CREATE TABLE Borrower (
   cust_name VARCHAR(50),
   loan_no INT,
   PRIMARY KEY (cust_name, loan_no),
   FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
   FOREIGN KEY (loan_no) REFERENCES Loan(loan_no)
);

INSert
INSERT INTO Branch VALUES ('Wadia College', 'Pune', 5000000);
INSERT INTO Branch VALUES ('Shivaji Nagar', 'Pune', 3000000);
INSERT INTO Branch VALUES ('Karve Nagar', 'Pune', 2000000);


INSERT INTO Account VALUES (101, 'Wadia College', 15000);
INSERT INTO Account VALUES (102, 'Shivaji Nagar', 20000);
INSERT INTO Account VALUES (103, 'Karve Nagar', 12000);

INSERT INTO Customer VALUES ('Amit', 'JM Road', 'Pune');
INSERT INTO Customer VALUES ('Sneha', 'FC Road', 'Pune');
INSERT INTO Customer VALUES ('Rahul', 'Karve Road', 'Pune');

INSERT INTO Depositor VALUES ('Amit', 101);
INSERT INTO Depositor VALUES ('Sneha', 102);

INSERT INTO Loan VALUES (201, 'Wadia College', 15000);
INSERT INTO Loan VALUES (202, 'Shivaji Nagar', 10000);
INSERT INTO Loan VALUES (203, 'Wadia College', 20000);

INSERT INTO Borrower VALUES ('Amit', 201);
INSERT INTO Borrower VALUES ('Rahul', 203);


1. Find the names of all branches in loan relation.
SELECT DISTINCT branch_name
FROM Loan;

2. Find all loan numbers for loans made at ‘Wadia College’ Branch
with loan amount > 12000.
SELECT loan_no
FROM Loan
WHERE branch_name = 'Wadia College'
AND amount > 12000;
```

3. Find all customers who have a loan from bank. Find their names,loan_no and loan amount.

```
SELECT b.cust_name, l.loan_no, l.amount
FROM Borrower b
JOIN Loan l ON b.loan_no = l.loan_no;
```

4. List all customers in alphabetical order who have loan from 'Wadia College' branch.

5. Display distinct cities of branch.

```
SELECT DISTINCT b.cust_name
FROM Borrower b
JOIN Loan l ON b.loan_no = l.loan_no
WHERE l.branch_name = 'Wadia College'
ORDER BY b.cust_name ASC;
```

S2

Consider following Relation

Account (Acc_no, branch_name,balance)

Branch(branch_name,branch_city,assets)

Customer(cust_name,cust_street,cust_city)

Depositor(cust_name,acc_no)

Loan(loan_no,branch_name,amount)

Borrower(cust_name,loan_no)

Create above tables with appropriate constraints like primary key, foreign key, not null etc.

```
CREATE TABLE Branch (
   branch_name VARCHAR(50) PRIMARY KEY,
   branch_city VARCHAR(50) NOT NULL,
   assets DECIMAL(12,2)
);

CREATE TABLE Account (
   acc_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   balance DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);

CREATE TABLE Customer (
   cust_name VARCHAR(50) PRIMARY KEY,
   cust_street VARCHAR(100),
   cust_city VARCHAR(50)
```

```
);

CREATE TABLE Depositor (
    cust_name VARCHAR(50),
    acc_no INT,
    PRIMARY KEY (cust_name, acc_no),
    FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
    FOREIGN KEY (acc_no) REFERENCES Account(acc_no)
);

CREATE TABLE Loan (
    loan_no INT PRIMARY KEY,
    branch_name VARCHAR(50) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);

CREATE TABLE Borrower (
    cust_name VARCHAR(50),
    loan_no INT,
    PRIMARY KEY (cust_name, loan_no),
    FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
    FOREIGN KEY (loan_no) REFERENCES Loan(loan_no)
);

INSERT INTO Branch VALUES ('Wadia College', 'Pune', 5000000);
INSERT INTO Branch VALUES ('Shivaji Nagar', 'Pune', 3000000);
INSERT INTO Branch VALUES ('Deccan', 'Pune', 4000000);

INSERT INTO Customer VALUES ('Amit', 'FC Road', 'Pune');
INSERT INTO Customer VALUES ('Neha', 'JM Road', 'Pune');
INSERT INTO Customer VALUES ('Rahul', 'Baner', 'Pune');
INSERT INTO Customer VALUES ('Sneha', 'Kothrud', 'Pune');
INSERT INTO Customer VALUES ('Rohit', 'Aundh', 'Pune');

INSERT INTO Account VALUES (101, 'Wadia College', 20000);
INSERT INTO Account VALUES (102, 'Shivaji Nagar', 15000);
INSERT INTO Account VALUES (103, 'Deccan', 30000);
INSERT INTO Account VALUES (104, 'Wadia College', 25000);

INSERT INTO Depositor VALUES ('Amit', 101);
INSERT INTO Depositor VALUES ('Neha', 102);
INSERT INTO Depositor VALUES ('Rahul', 103);
INSERT INTO Depositor VALUES ('Sneha', 104);
```

```
INSERT INTO Loan VALUES (201, 'Wadia College', 50000);
INSERT INTO Loan VALUES (202, 'Shivaji Nagar', 60000);
INSERT INTO Loan VALUES (203, 'Deccan', 70000);

INSERT INTO Borrower VALUES ('Amit', 201);
INSERT INTO Borrower VALUES ('Rahul', 203);
INSERT INTO Borrower VALUES ('Rohit', 202);
```

1. Find all customers who have both account and loan at bank.

```
SELECT DISTINCT d.cust_name
FROM Depositor d
INNER JOIN Borrower b
ON d.cust_name = b.cust_name;
```

2. Find all customers who have an account or loan or both at bank.

```
SELECT cust_name FROM Depositor
UNION
SELECT cust_name FROM Borrower;
```

3. Find all customers who have account but no loan at the bank.

```
SELECT cust_name
FROM Depositor
WHERE cust_name NOT IN (
   SELECT cust_name FROM Borrower
);
```

4. Find average account balance at ‘Wadia College’ branch.

```
SELECT AVG(balance) AS avg_balance
FROM Account
WHERE branch_name = 'Wadia College';
```

5. Find no. of depositors at each branch

```
SELECT A.branch_name, COUNT(DISTINCT D.cust_name) AS num_depositors
FROM Account A
JOIN Depositor D ON A.acc_no = D.acc_no
GROUP BY A.branch_name;
```

p10

Trigger: Write a after trigger for Insert, update and delete event

considering following requirement:
Emp(Emp_no, Emp_name, Emp_salary)

```
CREATE TABLE Emp (
   Emp_no INT PRIMARY KEY,
   Emp_name VARCHAR(50),
   Emp_salary DECIMAL(10,2)
);

CREATE TABLE Tracking (
   Emp_no INT,
   Emp_salary DECIMAL(10,2)
);
```

a) Trigger should be initiated when salary tried to be inserted is less than Rs.50,000/-

```
DELIMITER $$

CREATE TRIGGER trg_after_insert_emp
AFTER INSERT ON Emp
FOR EACH ROW
BEGIN
   IF NEW.Emp_salary < 50000 THEN
      INSERT INTO Tracking(Emp_no, Emp_salary)
      VALUES (NEW.Emp_no, NEW.Emp_salary);
   END IF;
END$$

DELIMITER ;
```

b) Trigger should be initiated when salary tried to be updated for value less than Rs. 50,000/-

```
DELIMITER $$

CREATE TRIGGER trg_after_update_emp
AFTER UPDATE ON Emp
FOR EACH ROW
BEGIN
   IF NEW.Emp_salary < 50000 THEN
      INSERT INTO Tracking(Emp_no, Emp_salary)
      VALUES (NEW.Emp_no, NEW.Emp_salary);
   END IF;
END$$

DELIMITER ;
```

Also the new values expected to be inserted will be stored in new table Tracking(Emp_no,Emp_salary).

```
DELIMITER $$

CREATE TRIGGER trg_after_delete_emp
AFTER DELETE ON Emp
FOR EACH ROW
BEGIN
   IF OLD.Emp_salary < 50000 THEN
      INSERT INTO Tracking(Emp_no, Emp_salary)
      VALUES (OLD.Emp_no, OLD.Emp_salary);
   END IF;
END$$

DELIMITER ;
```

S3

Consider following Relation

Account (Acc_no, branch_name,balance)

Branch(branch_name,branch_city,assets)

Customer(cust_name,cust_street,cust_city)

Depositor(cust_name,acc_no)

Loan(loan_no,branch_name,amount)

Borrower(cust_name,loan_no)

Create above tables with appropriate constraints like primary key, foreign key, not null etc.

```
CREATE TABLE Branch (
   branch_name VARCHAR(50) PRIMARY KEY,
   branch_city VARCHAR(50) NOT NULL,
   assets DECIMAL(12,2)
);

CREATE TABLE Account (
   acc_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   balance DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);

CREATE TABLE Customer (
   cust_name VARCHAR(50) PRIMARY KEY,
```

```
    cust_street VARCHAR(100),
    cust_city VARCHAR(50)
);

CREATE TABLE Depositor (
    cust_name VARCHAR(50),
    acc_no INT,
    PRIMARY KEY (cust_name, acc_no),
    FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
    FOREIGN KEY (acc_no) REFERENCES Account(acc_no)
);

CREATE TABLE Loan (
    loan_no INT PRIMARY KEY,
    branch_name VARCHAR(50) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)
);

CREATE TABLE Borrower (
    cust_name VARCHAR(50),
    loan_no INT,
    PRIMARY KEY (cust_name, loan_no),
    FOREIGN KEY (cust_name) REFERENCES Customer(cust_name),
    FOREIGN KEY (loan_no) REFERENCES Loan(loan_no)
);
```

1. Find the branches where average account balance > 15000.

```
SELECT branch_name, AVG(balance) AS avg_balance
FROM Account
GROUP BY branch_name
HAVING AVG(balance) > 15000;
```

2. Find number of tuples in customer relation.

```
SELECT COUNT(*) AS total_customers
FROM Customer;
```

3. Calculate total loan amount given by bank.

```
SELECT SUM(amount) AS total_loan
FROM Loan;
```

4. Delete all loans with loan amount between 1300 and 1500.

```
DELETE FROM Loan
```

```
WHERE amount BETWEEN 1300 AND 1500;
```

5. Find the average account balance at each branch

```
SELECT branch_name, AVG(balance) AS avg_balance
FROM Account
GROUP BY branch_name;
```

6. Find name of Customer and city where customer name starts with Letter P.

```
SELECT cust_name, cust_city
FROM Customer
WHERE cust_name LIKE 'P%';
```

S4

SQL Queries:

Create following tables with suitable constraints (primary key, foreign key, not null etc).

Insert record and solve the following queries:

Create table Cust_Master(Cust_no, Cust_name, Cust_addr)

Create table Order(Order_no, Cust_no, Order_date, Qty_Ordered)

Create Product (Product_no, Product_name, Order_no)

```
CREATE TABLE Cust_Master (
   Cust_no VARCHAR(10) PRIMARY KEY,
   Cust_name VARCHAR(50) NOT NULL,
   Cust_addr VARCHAR(100)
);

CREATE TABLE Orders (
   Order_no INT PRIMARY KEY,
   Cust_no VARCHAR(10),
   Order_date DATE,
   Qty_Ordered INT NOT NULL,
   FOREIGN KEY (Cust_no) REFERENCES Cust_Master(Cust_no)
);

CREATE TABLE Product (
   Product_no INT PRIMARY KEY,
   Product_name VARCHAR(50) NOT NULL,
   Order_no INT,
   FOREIGN KEY (Order_no) REFERENCES Orders(Order_no)
);
```

```
-- Customers
INSERT INTO Cust_Master VALUES ('C1001','Amit','Pune');
INSERT INTO Cust_Master VALUES ('C1002','Raj','Banglore');
INSERT INTO Cust_Master VALUES ('C1003','Pankaj','Manglore');
INSERT INTO Cust_Master VALUES ('C1004','Neha','Mumbai');
INSERT INTO Cust_Master VALUES ('C1005','Aakash','Banglore');
INSERT INTO Cust_Master VALUES ('C1007','Sagar','Manglore');
INSERT INTO Cust_Master VALUES ('C1008','Ankit','Delhi');

-- Orders
INSERT INTO Orders VALUES (1,'C1001','2024-01-10',2);
INSERT INTO Orders VALUES (2,'C1002','2024-02-12',5);
INSERT INTO Orders VALUES (3,'C1005','2024-03-15',3);
INSERT INTO Orders VALUES (4,'C1007','2024-04-18',4);
INSERT INTO Orders VALUES (5,'C1008','2024-05-20',6);

-- Products
INSERT INTO Product VALUES (101,'Laptop',1);
INSERT INTO Product VALUES (102,'Mobile',2);
INSERT INTO Product VALUES (103,'Tablet',3);
INSERT INTO Product VALUES (104,'Camera',4);
INSERT INTO Product VALUES (105,'Headphones',5);
```

1. List names of customers having 'A' as second letter in their Name.

```
SELECT Cust_name
FROM Cust_Master
WHERE Cust_name LIKE '_A%';
```

2. Display order from Customer no C1002, C1005, C1007 and C1008

```
SELECT *
FROM Orders
WHERE Cust_no IN ('C1002','C1005','C1007','C1008');
```

3. List Clients who stay in either 'Banglore or 'Manglore'

```
SELECT *
FROM Cust_Master
WHERE Cust_addr IN ('Banglore','Manglore');
```

4. Display name of customers& the product_name they have purchase

```
SELECT C.Cust_name, P.Product_name
FROM Cust_Master C
```

```
JOIN Orders O ON C.Cust_no = O.Cust_no
JOIN Product P ON O.Order_no = P.Order_no;
```

5. Create view View1 consisting of Cust_name, Product_name.

```
CREATE VIEW View1 AS
SELECT C.Cust_name, P.Product_name
FROM Cust_Master C
JOIN Orders O ON C.Cust_no = O.Cust_no
JOIN Product P ON O.Order_no = P.Order_no;
```

6. Disply product_name and quantity purchase by each customer

```
SELECT C.Cust_name, P.Product_name, O.Qty_Ordered
FROM Cust_Master C
JOIN Orders O ON C.Cust_no = O.Cust_no
JOIN Product P ON O.Order_no = P.Order_no;
```

7. Perform different joint operation.

```
SELECT C.Cust_name, O.Order_no
FROM Cust_Master C
INNER JOIN Orders O
ON C.Cust_no = O.Cust_no;
```

P1

Write a PL/SQL code block to calculate the area of a circle for a value of radius varying from 5 to 9. Store the radius and the corresponding values of calculated area in an empty table named areas, consisting of two columns, radius and area.

```
CREATE TABLE areas (
   radius NUMBER,
   area NUMBER
);

DECLARE
   r NUMBER;
   a NUMBER;
BEGIN
   r := 5;

   WHILE r <= 9 LOOP
      a := 3.14 * r * r;  -- Area formula: πr²
```

```
        INSERT INTO areas(radius, area)
        VALUES (r, a);

        r := r + 1;
    END LOOP;

    COMMIT;
END;
/
```

P3
Write a PL/SQL block of code using Cursor that will merge the data available in the newly created table N_Roll Call with the data available in the table O_RollCall. If the data in the first table already exist in the second table, then that data should be skipped.

```
-- Old table
CREATE TABLE O_RollCall (
    Roll_no INT PRIMARY KEY,
    Name VARCHAR2(50)
);

-- New table
CREATE TABLE N_RollCall (
    Roll_no INT,
    Name VARCHAR2(50)
);

DECLARE
    -- Cursor to fetch data from new table
    CURSOR c1 IS
        SELECT Roll_no, Name FROM N_RollCall;

    v_roll N_RollCall.Roll_no%TYPE;
    v_name N_RollCall.Name%TYPE;

    v_count NUMBER;

BEGIN
    OPEN c1;

    LOOP
        FETCH c1 INTO v_roll, v_name;
```

```
        EXIT WHEN c1%NOTFOUND;

        -- Check if record already exists in old table
        SELECT COUNT(*)
        INTO v_count
        FROM O_RollCall
        WHERE Roll_no = v_roll;

        -- If not exists, then insert
        IF v_count = 0 THEN
           INSERT INTO O_RollCall (Roll_no, Name)
           VALUES (v_roll, v_name);
        END IF;

    END LOOP;

    CLOSE c1;

    COMMIT;
END;
/
```

S5

Consider following Relation
Employee(emp_id,employee_name,street,city)
Works(employee_name,company_name,salary)
Company(company_name,city)
Manages(employee_name,manager_name)
Create above tables with appropriate constraints like primary key, foreign key, not null etc.

```
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    employee_name VARCHAR(50) UNIQUE,
    street VARCHAR(100),
    city VARCHAR(50)
);

CREATE TABLE Company (
    company_name VARCHAR(50) PRIMARY KEY,
    city VARCHAR(50)
);
```

```
CREATE TABLE Works (
   employee_name VARCHAR(50),
   company_name VARCHAR(50),
   salary DECIMAL(10,2) NOT NULL,
   PRIMARY KEY (employee_name),
   FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),
   FOREIGN KEY (company_name) REFERENCES Company(company_name)
);

CREATE TABLE Manages (
   employee_name VARCHAR(50),
   manager_name VARCHAR(50),
   PRIMARY KEY (employee_name),
   FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
);

INSERT INTO Company VALUES ('TCS', 'Mumbai');
INSERT INTO Company VALUES ('InfoSys', 'Pune');
INSERT INTO Company VALUES ('TechM', 'Hyderabad');
INSERT INTO Company VALUES ('Wipro', 'Bangalore');

INSERT INTO Employee VALUES (1, 'Amit', 'FC Road', 'Pune');
INSERT INTO Employee VALUES (2, 'Neha', 'JM Road', 'Mumbai');
INSERT INTO Employee VALUES (3, 'Rahul', 'Baner', 'Pune');
INSERT INTO Employee VALUES (4, 'Sneha', 'Kothrud', 'Pune');
INSERT INTO Employee VALUES (5, 'Rohit', 'Aundh', 'Mumbai');
INSERT INTO Employee VALUES (6, 'Pooja', 'Shivaji Nagar', 'Pune');

INSERT INTO Manages VALUES ('Amit', 'Neha');
INSERT INTO Manages VALUES ('Rahul', 'Sneha');
INSERT INTO Manages VALUES ('Rohit', 'Amit');
```

1. Find the names of all employees who work for ‘TCS’.

```
SELECT employee_name
FROM Works
WHERE company_name = 'TCS';
```

2. Find the names and company names of all employees sorted in ascending order of company name and descending order of employee names of that company.

```
SELECT employee_name, company_name
FROM Works
```

```
ORDER BY company_name ASC, employee_name DESC;
```

3. Change the city of employee working with InfoSys to ‘Bangalore’

```
UPDATE Employee
SET city = 'Bangalore'
WHERE employee_name IN (
   SELECT employee_name
   FROM Works
   WHERE company_name = 'InfoSys'
);
```

4. Find the names, street address, and cities of residence for all employees who work for 'TechM' and earn more than $10,000.

```
SELECT E.employee_name, E.street, E.city
FROM Employee E
JOIN Works W ON E.employee_name = W.employee_name
WHERE W.company_name = 'TechM'
AND W.salary > 10000;
```

5. Add Column Asset to Company table.

```
ALTER TABLE Company
ADD Asset DECIMAL(12,2);
```

S6

Consider following Relation

Employee(emp_id,employee_name,street,city)

Works(employee_name,company_name,salary)

Company(company_name,city)

Manages(employee_name,manager_name)

Create above tables with appropriate constraints like primary key, foreign key, not null etc.

```
CREATE TABLE Employee (
   emp_id INT PRIMARY KEY,
   employee_name VARCHAR(50) UNIQUE,
   street VARCHAR(100),
   city VARCHAR(50)
);

CREATE TABLE Company (
   company_name VARCHAR(50) PRIMARY KEY,
   city VARCHAR(50)
);
```

```
CREATE TABLE Works (
   employee_name VARCHAR(50),
   company_name VARCHAR(50),
   salary DECIMAL(10,2) NOT NULL,
   PRIMARY KEY (employee_name),
   FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),
   FOREIGN KEY (company_name) REFERENCES Company(company_name)
);

CREATE TABLE Manages (
   employee_name VARCHAR(50),
   manager_name VARCHAR(50),
   PRIMARY KEY (employee_name),
   FOREIGN KEY (employee_name) REFERENCES Employee(employee_name)
);
```

1. Change the city of employee working with InfoSys to ‘Bangalore’

```
UPDATE Employee
SET city = 'Bangalore'
WHERE employee_name IN (
   SELECT employee_name
   FROM Works
   WHERE company_name = 'InfoSys'
);
```

2. Find the names of all employees who earn more than the average

```
SELECT W.employee_name
FROM Works W
WHERE W.salary > (
   SELECT AVG(W2.salary)
   FROM Works W2
   WHERE W2.company_name = W.company_name
);
```

salary of all employees of their company. Assume that all people work for at most one company.

```
SELECT E.employee_name, E.street, E.city
FROM Employee E
JOIN Works W ON E.employee_name = W.employee_name
WHERE W.company_name = 'TechM'
AND W.salary > 10000;
```

3. Find the names, street address, and cities of residence for all employees who work for 'TechM' and earn more than $10,000.

```
ALTER TABLE Manages
RENAME TO Management;
```

4. Change name of table Manages to Management.

```
ALTER TABLE Manages
RENAME TO Management;
```

5. Create Simple and Unique index on employee table.

6. Display index Information

P8

Write a Row Level Before and After Trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_Audit table.

```
CREATE TABLE Library (
   Book_ID NUMBER PRIMARY KEY,
   Book_Name VARCHAR2(50),
   Author VARCHAR2(50),
   Price NUMBER
);

CREATE TABLE Library_Audit (
   Audit_ID NUMBER GENERATED ALWAYS AS IDENTITY,
   Book_ID NUMBER,
   Book_Name VARCHAR2(50),
   Author VARCHAR2(50),
   Price NUMBER,
   Operation VARCHAR2(10),   -- UPDATE or DELETE
   Action_Date DATE
);

INSERT INTO Library VALUES (1, 'DBMS', 'Korth', 500);
INSERT INTO Library VALUES (2, 'OS', 'Galvin', 650);
INSERT INTO Library VALUES (3, 'CN', 'Tanenbaum', 700);
COMMIT;
```

BEFORE UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_library_before_update
BEFORE UPDATE ON Library
FOR EACH ROW
BEGIN
```

```
    INSERT INTO Library_Audit (
        Book_ID, Book_Name, Author, Price, Operation, Action_Date
    )
    VALUES (
        :OLD.Book_ID,
        :OLD.Book_Name,
        :OLD.Author,
        :OLD.Price,
        'UPDATE',
        SYSDATE
    );
END;
/
```

BEFORE DELETE Trigger

```
CREATE OR REPLACE TRIGGER trg_library_before_delete
BEFORE DELETE ON Library
FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit (
        Book_ID, Book_Name, Author, Price, Operation, Action_Date
    )
    VALUES (
        :OLD.Book_ID,
        :OLD.Book_Name,
        :OLD.Author,
        :OLD.Price,
        'DELETE',
        SYSDATE
    );
END;
/
```

AFTER UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_library_after_update
AFTER UPDATE ON Library
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Record updated successfully');
END;
/
```

```
UPDATE Library SET Price = 550 WHERE Book_ID = 1;

DELETE FROM Library WHERE Book_ID = 2;

SELECT * FROM Library_Audit;
```

P9 Trigger: Create a row level trigger for the CUSTOMERS table that would fire INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old values and new values.

```
CREATE TABLE CUSTOMERS (
   ID INT PRIMARY KEY,
   NAME VARCHAR(50),
   AGE INT,
   ADDRESS VARCHAR(100),
   SALARY INT
);

INSERT INTO CUSTOMERS VALUES
(1,'Pallavi',21,'Pune',20000),
(2,'Ravi',22,'Mumbai',25000),
(3,'Neha',23,'Nashik',30000);

DELIMITER //

CREATE TRIGGER trg_insert_salary
AFTER INSERT ON CUSTOMERS
FOR EACH ROW
BEGIN
   SELECT 'INSERT OPERATION' AS MESSAGE,
        NEW.SALARY AS NEW_SALARY,
        NEW.SALARY AS SALARY_DIFFERENCE;
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER trg_update_salary
AFTER UPDATE ON CUSTOMERS
FOR EACH ROW
BEGIN
   SELECT 'UPDATE OPERATION' AS MESSAGE,
        OLD.SALARY AS OLD_SALARY,
```

```
        NEW.SALARY AS NEW_SALARY,
        (NEW.SALARY - OLD.SALARY) AS SALARY_DIFFERENCE;
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER trg_delete_salary
AFTER DELETE ON CUSTOMERS
FOR EACH ROW
BEGIN
   SELECT 'DELETE OPERATION' AS MESSAGE,
        OLD.SALARY AS OLD_SALARY,
        (0 - OLD.SALARY) AS SALARY_DIFFERENCE;
END //

DELIMITER ;

INSERT INTO CUSTOMERS VALUES (4,'Amit',24,'Delhi',35000);

UPDATE CUSTOMERS SET SALARY = 40000 WHERE ID = 1;

DELETE FROM CUSTOMERS WHERE ID = 2;

SELECT * FROM CUSTOMERS;
```

S7

Consider following Relation

Account (Acc_no, branch_name,balance)

Branch(branch_name,branch_city,assets)

Customer(cust_name,cust_street,cust_city)

Depositor(cust_name,acc_no)

Loan(loan_no,branch_name,amount)

Borrower(cust_name,loan_no)

Execute the following query:

1. Create a View1 to display List all customers in alphabetical order who have loan from Pune_Station branch.
2. Create View2 on branch table by selecting any two columns and perform insert update delete operations.
3. Create View3 on borrower and depositor table by selecting any one column from each table perform insert update delete operations.

4. Create Union of left and right joint for all customers who have
an account or loan or both at bank
5. Create Simple and Unique index.
6. Display index Information.

CREATE TABLE Account (Acc_no INT PRIMARY KEY, branch_name VARCHAR(50), balance INT);

CREATE TABLE Branch (branch_name VARCHAR(50) PRIMARY KEY, branch_city VARCHAR(50), assets INT NULL);

CREATE TABLE Customer (cust_name VARCHAR(50) PRIMARY KEY, cust_street VARCHAR(50), cust_city VARCHAR(50));

CREATE TABLE Depositor cust_name VARCHAR(50) acc_no INT);

CREATE TABLE Loan (loan_no INT PRIMARY KEY,bbbranch_name VARCHAR(50), amount INT);

CREATE TABLE Loan (loan_no INT PRIMARY KEY, branch_name VARCHAR(50), amount INT);

INSERT INTO Branch VALUES('Pune_Station','Pune',500000), ('Shivaji_Nagar','Pune',400000),('Mumbai_Central','Mumbai',700000);

INSERT INTO Account VALUES(101,'Pune_Station',10000), (102,'Shivaji_Nagar',15000),(103,'Mumbai_Central',20000);

INSERT INTO Customer VALUES('Amit','MG Road','Pune'), ('Bhavna','FC Road','Pune'),('Chetan','Dadar','Mumbai'),('Deepa','Camp','Pune');

INSERT INTO Depositor VALUES('Amit',101),('Bhavna',102),('Chetan',103);

INSERT INTO Loan VALUES(201,'Pune_Station',50000),(202,'Mumbai_Central',70000);

INSERT INTO Borrower VALUES('Deepa',201),('Chetan',202);

1.
```
CREATE VIEW View1 AS
SELECT B.cust_name
FROM Borrower B
JOIN Loan L
ON B.loan_no = L.loan_no
WHERE L.branch_name = 'Pune_Station'
```

```
ORDER BY B.cust_name;

SELECT * FROM View1;

2
CREATE VIEW View2 AS
SELECT branch_name, branch_city
FROM Branch;

SELECT * FROM View2;

INSERT INTO View2 VALUES ('Kothrud_Branch','Pune');

SELECT * FROM Branch;

UPDATE View2
SET branch_city = 'Mumbai'
WHERE branch_name = 'Kothrud_Branch';

SELECT * FROM Branch;

DELETE FROM View2 WHERE branch_name = 'Kothrud_Branch';

SELECT * FROM Branch;

3.
CREATE VIEW View3 AS
SELECT B.cust_name, D.acc_no
FROM Borrower B
JOIN Depositor D
ON B.cust_name = D.cust_name;

SELECT * FROM View3;

INSERT INTO Depositor VALUES ('Deepa',101);

UPDATE Depositor SET acc_no = 102 WHERE cust_name = 'Deepa';

DELETE FROM DepositornWHERE cust_name = 'Deepa';

SELECT * FROM View3;

4
SELECT C.cust_name, D.acc_no, B.loan_no
```

```
FROM Customer C
LEFT JOIN Depositor D
ON C.cust_name = D.cust_name
LEFT JOIN Borrower B
ON C.cust_name = B.cust_name
UNION
SELECT C.cust_name, D.acc_no, B.loan_no
FROM Customer C
RIGHT JOIN Borrower B
ON C.cust_name = B.cust_name
LEFT JOIN Depositor D
ON C.cust_name = D.cust_name;
```

5

```
CREATE INDEX idx_branchname ON Account(branch_name);

CREATE UNIQUE INDEX idx_unique_custname ON Customer(cust_name);
```

6

```
SHOW INDEX FROM Account;
SHOW INDEX FROM Customer;
```

S8

Consider following Relation:
Companies (comp_id, name, cost, year)
Orders (comp_id, domain, quantity)
Execute the following query:
1. Find names, costs, domains and quantities for companies using inner join.
2. Find names, costs, domains and quantities for companies using left outer join.
3. Find names, costs, domains and quantities for companies using right outer join.
4. Find names, costs, domains and quantities for companies using Union operator.
5. Create View View1 by selecting both tables to show company name and quantities.
6. Create View View2 by selecting any two columns and perform insert update delete operations.
7. Display content of View1, View2.

```
CREATE TABLE Companies (
   comp_id INT PRIMARY KEY,
   name VARCHAR(50),
   cost INT,
```

```
    year INT
);

CREATE TABLE Orders (
    comp_id INT,
    domain VARCHAR(50),
    quantity INT
);

INSERT INTO Companies VALUES
(1,'Infosys',50000,2010),
(2,'TCS',70000,2012),
(3,'Wipro',60000,2015),
(4,'TechMahindra',80000,2018);

INSERT INTO Orders VALUES
(1,'Software',10),
(2,'Networking',20),
(3,'Database',15),
(5,'Cloud',25);

1
SELECT C.name, C.cost, O.domain, O.quantity
FROM Companies C
INNER JOIN Orders O
ON C.comp_id = O.comp_id;

2
SELECT C.name, C.cost, O.domain, O.quantity
FROM Companies C
LEFT JOIN Orders O
ON C.comp_id = O.comp_id;

3
SELECT C.name, C.cost, O.domain, O.quantity
FROM Companies C
RIGHT JOIN Orders O
ON C.comp_id = O.comp_id;

4
SELECT C.name, C.cost, O.domain, O.quantity
FROM Companies C
LEFT JOIN Orders O
ON C.comp_id = O.comp_id
```

```
UNION

SELECT C.name, C.cost, O.domain, O.quantity
FROM Companies C
RIGHT JOIN Orders O
ON C.comp_id = O.comp_id;
```

5

```
CREATE VIEW View1 AS
SELECT C.name, O.quantity
FROM Companies C
JOIN Orders O
ON C.comp_id = O.comp_id;

SELECT * FROM View1;
```

6

```
CREATE VIEW View2 AS
SELECT comp_id, name
FROM Companies;

SELECT * FROM View2;

INSERT INTO View2 VALUES (6,'Capgemini');

UPDATE View2
SET name = 'Capgemini_India'
WHERE comp_id = 6;

DELETE FROM View2
WHERE comp_id = 6;
```

7

```
SELECT * FROM View1;
SELECT * FROM View2;
```

P5

Write a PL/SQL Block to increase the salary of employees by 10% of existing salary, who are having salary less than average salary of organization, whenever such salary updates take place, a record for same is maintained in the increment_salary table.

emp(emp_no, salary)

increment_salary(emp_no, salary)

```
CREATE DATABASE emp_db;
USE emp_db;

CREATE TABLE emp (
   emp_no INT PRIMARY KEY,
   salary INT
);

CREATE TABLE increment_salary (
   emp_no INT,
   salary INT
);

INSERT INTO emp VALUES
(1,20000),
(2,30000),
(3,15000),
(4,50000),
(5,10000);

DELIMITER //

CREATE TRIGGER trg_increment_record
AFTER UPDATE ON emp
FOR EACH ROW
BEGIN
   INSERT INTO increment_salary VALUES (NEW.emp_no, NEW.salary);
END //

CREATE PROCEDURE increase_emp_salary()
BEGIN
   DECLARE avg_sal DECIMAL(10,2);

   SELECT AVG(salary) INTO avg_sal
   FROM emp;

   UPDATE emp
   SET salary = salary + (salary * 0.10)
   WHERE salary < avg_sal;
END //

DELIMITER ;
```

```
CALL increase_emp_salary();

SELECT * FROM emp;
SELECT * FROM increment_salary;
```

S9

SQL Queries

Create following tables with suitable constraints. Insert data and solve the following queries:

CUSTOMERS(CNo, Cname, Ccity, CMobile)

ITEMS(INo, Iname, Itype, Iprice, Icount)

PURCHASE(PNo, Pdate, Pquantity, Cno, INo)

1. List all stationary items with price between 400/- to 1000/-
2. Change the mobile number of customer “Gopal”
3. Display the item with maximum price
4. Display all purchases sorted from the most recent to the oldest
5. Count the number of customers in every city
6. Display all purchased quantity of Customer Maya
7. Create view which shows Iname, Price and Count of all stationary items in descending order of price.

```
CREATE TABLE CUSTOMERS (
   CNo INT PRIMARY KEY,
   Cname VARCHAR(50),
   Ccity VARCHAR(50),
   CMobile VARCHAR(15)
);

CREATE TABLE ITEMS (
   INo INT PRIMARY KEY,
   Iname VARCHAR(50),
   Itype VARCHAR(30),
   Iprice INT,
   Icount INT
);

CREATE TABLE PURCHASE (
   PNo INT PRIMARY KEY,
   Pdate DATE,
   Pquantity INT,
   CNo INT,
   INo INT,
   FOREIGN KEY (CNo) REFERENCES CUSTOMERS(CNo),
   FOREIGN KEY (INo) REFERENCES ITEMS(INo)
```

```
);

INSERT INTO CUSTOMERS VALUES
(1,'Maya','Pune','9876543210'),
(2,'Gopal','Mumbai','9988776655'),
(3,'Rina','Pune','9123456789'),
(4,'Amit','Nashik','9012345678');

INSERT INTO ITEMS VALUES
(101,'Notebook','Stationary',500,20),
(102,'Pen','Stationary',450,50),
(103,'Printer','Electronics',3000,5),
(104,'File','Stationary',800,15),
(105,'Mouse','Electronics',700,10);

INSERT INTO PURCHASE VALUES
(1001,'2024-01-10',5,1,101),
(1002,'2024-02-15',2,2,103),
(1003,'2024-03-20',10,1,102),
(1004,'2024-04-05',3,3,104),
(1005,'2024-05-01',1,4,105);

1
SELECT * FROM ITEMS
WHERE Itype='Stationary'
AND Iprice BETWEEN 400 AND 1000;

2
UPDATE CUSTOMERS
SET CMobile='9999999999'
WHERE Cname='Gopal';

3
SELECT * FROM ITEMS
WHERE Iprice = (SELECT MAX(Iprice) FROM ITEMS);

4
SELECT * FROM PURCHASE
ORDER BY Pdate DESC;

5
SELECT Ccity, COUNT(*) AS Total_Customers
FROM CUSTOMERS
GROUP BY Ccity;
```

6

```
SELECT C.Cname, P.Pquantity
FROM CUSTOMERS C
JOIN PURCHASE P
ON C.CNo = P.CNo
WHERE C.Cname='Maya';
```

7

```
CREATE VIEW Stationary_View AS
SELECT Iname, Iprice, Icount
FROM ITEMS
WHERE Itype='Stationary'
ORDER BY Iprice DESC;

SELECT * FROM Stationary_View;
```

M1

Design and Develop MongoDB Queries using CRUD operations:

Create Employee collection by considering following Fields:

i. Name: Embedded Doc (FName, LName)
ii. Company Name: String
iii. Salary: Number
iv. Designation: String
v. Age: Number
vi. Expertise: Array
vii. DOB: String or Date
viii. Email id: String
ix. Contact: String
x. Address: Array of Embedded Doc (PAddr, LAddr)

Insert at least 5 documents in collection by considering above attribute and execute following queries:

1. Select all documents where the Designation field has the value "Programmer" and the value of the salary field is greater than 30000.
2. Creates a new document if no document in the employee collection contains
{Designation: "Tester", Company_name: "TCS", Age: 25}
3. Increase salary of each Employee working with “Infosys" 10000.
4. Finds all employees working with "TCS" and reduce their salary by 5000.
5. Return documents where Designation is not equal to "Tester".
6. Find all employee with Exact Match on an Array having Expertise: ['Mongodb','Mysql','Cassandra']

```
db.Employee.insertMany([
{
   Name:{FName:"Amit",LName:"Sharma"},
   Company_name:"Infosys",
   Salary:35000,
   Designation:"Programmer",
   Age:24,
   Expertise:["Java","Mysql","Mongodb"],
   DOB:"2001-02-10",
   Email_id:"amit@gmail.com",
   Contact:"9876543210",
   Address:[
      {PAddr:"Pune"},
      {LAddr:"Mumbai"}
   ]
},
{
   Name:{FName:"Rina",LName:"Patil"},
   Company_name:"TCS",
   Salary:28000,
   Designation:"Tester",
   Age:25,
   Expertise:["Testing","Mysql","Cassandra"],
   DOB:"2000-05-12",
   Email_id:"rina@gmail.com",
   Contact:"9988776655",
   Address:[
      {PAddr:"Nashik"},
      {LAddr:"Pune"}
   ]
},
{
   Name:{FName:"Maya",LName:"Joshi"},
   Company_name:"Infosys",
   Salary:40000,
   Designation:"Programmer",
   Age:26,
   Expertise:["Mongodb","Mysql","Cassandra"],
   DOB:"1999-08-15",
   Email_id:"maya@gmail.com",
   Contact:"9123456789",
   Address:[
      {PAddr:"Pune"},
      {LAddr:"Nagpur"}
```

```
    ]
},
{
    Name:{FName:"Gopal",LName:"Kale"},
    Company_name:"TCS",
    Salary:32000,
    Designation:"Developer",
    Age:27,
    Expertise:["Python","Mysql","Oracle"],
    DOB:"1998-09-09",
    Email_id:"gopal@gmail.com",
    Contact:"9012345678",
    Address:[
        {PAddr:"Mumbai"},
        {LAddr:"Pune"}
    ]
},
{
    Name:{FName:"Neha",LName:"Singh"},
    Company_name:"Wipro",
    Salary:45000,
    Designation:"Programmer",
    Age:28,
    Expertise:["Mongodb","Mysql","Cassandra"],
    DOB:"1997-03-20",
    Email_id:"neha@gmail.com",
    Contact:"9090909090",
    Address:[
        {PAddr:"Delhi"},
        {LAddr:"Pune"}
    ]
}
])

1
db.Employee.find({
    Designation:"Programmer",
    Salary:{$gt:30000}
})

2
db.Employee.updateOne(
    {
        Designation:"Tester",
```

```
      Company_name:"TCS",
      Age:25
   },
   {
      $setOnInsert:{
         Name:{FName:"Kiran",LName:"More"},
         Salary:30000,
         Expertise:["Manual","Automation"],
         DOB:"2001-01-01",
         Email_id:"kiran@gmail.com",
         Contact:"9898989898",
         Address:[
            {PAddr:"Satara"},
            {LAddr:"Pune"}
         ]
      }
   },
   {upsert:true}
)

3
db.Employee.updateMany(
   {Company_name:"Infosys"},
   {$inc:{Salary:10000}}
)

4
db.Employee.updateMany(
   {Company_name:"TCS"},
   {$inc:{Salary:-5000}}
)

5
db.Employee.find({
   Designation:{$ne:"Tester"}
})

6
db.Employee.find({
   Expertise:["Mongodb","Mysql","Cassandra"]
})
```

M2

Design and Develop MongoDB Queries using CRUD operations:
Create Employee collection by considering following Fields:
i. Name: Embedded Doc (FName, LName)
ii. Company Name: String
iii. Salary: Number
iv. Designation: String
v. Age: Number
vi. Expertise: Array
vii. DOB: String or Date
viii. Email id: String
ix. Contact: String
x. Address: Array of Embedded Doc (PAddr, LAddr)
Insert at least 5 documents in collection by considering above attribute and execute following queries:
1. Final name of Employee where age is less than 30 and salary more than 50000.
2. Creates a new document if no document in the employee collection contains
{Designation: "Tester", Company_name: "TCS", Age: 25}
3. Selects all documents in the collection where the field age has a value less than 30 or the value of the salary field is greater than 40000.
4. Find documents where Designation is not equal to "Developer".
5. Find _id, Designation, Address and Name from all documents where Company_name is "Infosys".
6. Display only FName and LName of all Employees

```
db.createCollection("Employee")

db.Employee.insertMany([
{
   Name:{FName:"Amit",LName:"Sharma"},
   Company_name:"Infosys",
   Salary:55000,
   Designation:"Programmer",
   Age:24,
   Expertise:["Java","Mysql","Mongodb"],
   DOB:"2001-02-10",
   Email_id:"amit@gmail.com",
   Contact:"9876543210",
   Address:[
      {PAddr:"Pune"},
      {LAddr:"Mumbai"}
   ]
```

```
},
{
   Name:{FName:"Rina",LName:"Patil"},
   Company_name:"TCS",
   Salary:28000,
   Designation:"Tester",
   Age:25,
   Expertise:["Testing","Mysql","Cassandra"],
   DOB:"2000-05-12",
   Email_id:"rina@gmail.com",
   Contact:"9988776655",
   Address:[
      {PAddr:"Nashik"},
      {LAddr:"Pune"}
   ]
},
{
   Name:{FName:"Maya",LName:"Joshi"},
   Company_name:"Infosys",
   Salary:60000,
   Designation:"Developer",
   Age:26,
   Expertise:["Mongodb","Mysql","Cassandra"],
   DOB:"1999-08-15",
   Email_id:"maya@gmail.com",
   Contact:"9123456789",
   Address:[
      {PAddr:"Pune"},
      {LAddr:"Nagpur"}
   ]
},
{
   Name:{FName:"Gopal",LName:"Kale"},
   Company_name:"TCS",
   Salary:35000,
   Designation:"Developer",
   Age:31,
   Expertise:["Python","Mysql","Oracle"],
   DOB:"1998-09-09",
   Email_id:"gopal@gmail.com",
   Contact:"9012345678",
   Address:[
      {PAddr:"Mumbai"},
      {LAddr:"Pune"}
```

```
    ]
},
{
    Name:{FName:"Neha",LName:"Singh"},
    Company_name:"Wipro",
    Salary:45000,
    Designation:"Programmer",
    Age:28,
    Expertise:["Mongodb","Mysql","Cassandra"],
    DOB:"1997-03-20",
    Email_id:"neha@gmail.com",
    Contact:"9090909090",
    Address:[
        {PAddr:"Delhi"},
        {LAddr:"Pune"}
    ]
}
])

1
db.Employee.find(
    {
        Age:{$lt:30},
        Salary:{$gt:50000}
    },
    {
        "Name.FName":1,
        _id:0
    }
)

2
db.Employee.updateOne(
    {
        Designation:"Tester",
        Company_name:"TCS",
        Age:25
    },
    {
        $setOnInsert:{
            Name:{FName:"Kiran",LName:"More"},
            Salary:30000,
            Expertise:["Manual","Automation"],
            DOB:"2001-01-01",
```

```
        Email_id:"kiran@gmail.com",
        Contact:"9898989898",
        Address:[
          {PAddr:"Satara"},
          {LAddr:"Pune"}
        ]
      }
  },
  {upsert:true}
)

3
db.Employee.find({
  $or:[
    {Age:{$lt:30}},
    {Salary:{$gt:40000}}
  ]
})

4
db.Employee.find({
  Designation:{$ne:"Developer"}
})

5
db.Employee.find(
  {Company_name:"Infosys"},
  {
    _id:1,
    Designation:1,
    Address:1,
    Name:1
  }
)

6
db.Employee.find(
  {},
  {
    "Name.FName":1,
    "Name.LName":1,
    _id:0
  }
)
```

db.Employee.find().pretty()

P4) Write a PL/SQL block for following requirements and handle the exceptions. Roll no. of students will be entered by the user. Attendance of roll no. entered by user will be checked in the Stud table. If attendance is less than 75% then display the message“Term not granted” and set the status in stud table as “Detained”. Otherwise display message “Term granted” and set the status in stud table as “Not Detained”. Student (Roll, Name, Attendance, Status)
Ans:

```
DECLARE
   v_roll       Stud.Roll%TYPE;
   v_attendance  Stud.Attendance%TYPE;
BEGIN
   v_roll := &Enter_Roll_No;
   SELECT Attendance
   INTO v_attendance
   FROM Stud
   WHERE Roll = v_roll;
   IF v_attendance < 75 THEN
      UPDATE Stud
      SET Status = 'Detained'
      WHERE Roll = v_roll;
      DBMS_OUTPUT.PUT_LINE('Term not granted');
   ELSE
      UPDATE Stud
      SET Status = 'Not Detained'
      WHERE Roll = v_roll;
      DBMS_OUTPUT.PUT_LINE('Term granted');
   END IF;
   COMMIT;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Student with given Roll No. not found');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

P7) Create a stored function titled 'Age_calc'. Accept the date of birth of a person as a parameter. Calculate the age of the person in years, months and days e.g. 3 years, 2months, 10 days.Return the age in years directly (with the help of Return statement). The months and days are to be returned indirectly in the form of OUT parameters.

Ans:

```
CREATE OR REPLACE FUNCTION Age_calc (
   p_dob   IN DATE,
   p_month OUT NUMBER,
   p_days  OUT NUMBER
) RETURN NUMBER
IS
   v_years NUMBER;
   v_total_months NUMBER;
BEGIN
   v_total_months := MONTHS_BETWEEN(SYSDATE, p_dob);
   v_years := FLOOR(v_total_months / 12);
   p_month := FLOOR(MOD(v_total_months, 12));
   p_days := TRUNC(SYSDATE - ADD_MONTHS(p_dob, FLOOR(v_total_months)));
   RETURN v_years;
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
      RETURN NULL;
END;
/
```

M3) Design and Develop MongoDB Queries using CRUD operations:
Create Employee collection by considering following Fields:
i. Emp_id : Number
ii. Name: Embedded Doc (FName, LName)
iii. Company Name: String
iv. Salary: Number
v. Designation: String
vi. Age: Number
vii. Expertise: Array
viii. DOB: String or Date
ix. Email id: String
x. Contact: String
xi. Address: Array of Embedded Doc (PAddr, LAddr)
Insert at least 5 documents in collection by considering above
attribute and execute following queries:

1. Creates a new document if no document in the employee collection
Contains
Ans:

```
db.Employee.updateOne(
  { Designation: "Tester", Company_name: "TCS", Age: 25 },
  {
   $setOnInsert: {
```

```
    Emp_id: 6,
    Name: { FName: "New", LName: "Employee" },
    Company_name: "TCS",
    Salary: 30000,
    Designation: "Tester",
    Age: 25,
    Expertise: ["Testing"],
    DOB: ISODate("2000-01-01"),
    Email_id: "new@gmail.com",
    Contact: "7777777777",
    Address: [
      { PAddr: { City: "Pune", Pin_code: "411001" },
        LAddr: { City: "Pune", Pin_code: "411001" } }
    ]
  }
 },
 { upsert: true }
)
```

{Designation: "Tester", Company_name: "TCS", Age: 25}

2. Finds all employees working with Company_name: "TCS" and increase their salary by 2000.

Ans:

```
db.Employee.updateMany(
 { Company_name: "TCS" },
 { $inc: { Salary: 2000 } }
)
```

3. Matches all documents where the value of the field Address is an embedded document that contains only the field city with the value "Pune" and the field Pin_code with the value "411001".

Ans:

```
db.Employee.find({
 Address: {
  $elemMatch: {
   "PAddr.City": "Pune",
   "PAddr.Pin_code": "411001"
  }
 }
})
```

4. Find employee details who are working as "Developer" or "Tester".

Ans:

```
db.Employee.find({
```

```
  $or: [
    { Designation: "Developer" },
    { Designation: "Tester" }
  ]
})
```

5. Drop Single documents where designation="Developer".
Ans:

```
db.Employee.deleteMany({
  Designation: "Developer"
})
```

6. Count number of documents in employee collection.
Ans:

```
db.Employee.countDocuments()
```

P6) Write a Stored Procedure namely proc_Grade for the categorization of student. If marks scored by students in examination is <=1500 and marks>=990 then student will be placed in distinction category if marks scored are between 989 and 900 categories is first class, if marks 899 and 825 category is Higher Second Class.
Write a PL/SQL block for using procedure created with above requirement.
Stud_Marks(name, total_marks),
Result (Roll,Name, Class)
Ans:

1)Stored procedure

```
CREATE OR REPLACE PROCEDURE proc_Grade (
    p_name   IN Stud_Marks.name%TYPE,
    p_class  OUT VARCHAR2
)
IS
    v_marks Stud_Marks.total_marks%TYPE;
BEGIN
    SELECT total_marks
    INTO v_marks
    FROM Stud_Marks
    WHERE name = p_name;
    IF v_marks BETWEEN 990 AND 1500 THEN
        p_class := 'Distinction';
    ELSIF v_marks BETWEEN 900 AND 989 THEN
```

```
        p_class := 'First Class';
    ELSIF v_marks BETWEEN 825 AND 899 THEN
        p_class := 'Higher Second Class';
    ELSE
        p_class := 'No Class';
    END IF;

    -- Insert result into Result table
    INSERT INTO Result (Roll, Name, Class)
    VALUES (NULL, p_name, p_class);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

2) PL/SQL Block to Call Procedure

```
DECLARE
    v_name  Stud_Marks.name%TYPE := '&Enter_Name';
    v_class VARCHAR2(50);
BEGIN
    -- Call procedure
    proc_Grade(v_name, v_class);

    -- Display result
    DBMS_OUTPUT.PUT_LINE('Student: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Class: ' || v_class);
END;
/
```

C1) Write a program to implement MongoDB database connectivity with PHP /python /Java Implement Database navigation CRUD operations (add, delete, edit etc.)

Ans:

Install dependency:

```
pip install pymongo
```

Python Program:

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["CompanyDB"]
collection = db["Employee"]
def add_employee(emp):
    collection.insert_one(emp)
    print("Employee added")
def view_employees():
    for emp in collection.find():
        print(emp)
def update_employee(emp_id, new_data):
    collection.update_one(
        {"Emp_id": emp_id},
        {"$set": new_data}
    )
    print("Employee updated")
def delete_employee(emp_id):
    collection.delete_one({"Emp_id": emp_id})
    print("Employee deleted")
if __name__ == "__main__":
    emp = {
        "Emp_id": 101,
        "Name": {"FName": "John", "LName": "Doe"},
        "Company_name": "TCS",
        "Salary": 30000,
        "Designation": "Developer",
        "Age": 25
    }

    add_employee(emp)             # Create
    view_employees()              # Read
    update_employee(101, {"Salary": 35000})  # Update
    delete_employee(101)          # Delete
```

C2) Implement MYSQL/Oracle database connectivity with PHP /python /Java
Implement Database navigation operations (add, delete, edit,).
Ans:
MySQL Table:

```
CREATE TABLE Employee (
    Emp_id INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary INT,
    Designation VARCHAR(50)
);
```

Python (Using mysql-connector-python):
pip install mysql-connector-python

Python Program:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="testdb"
)

cursor = conn.cursor()


def add_employee():
    emp_id = int(input("Enter ID: "))
    name = input("Enter Name: ")
    salary = int(input("Enter Salary: "))
    designation = input("Enter Designation: ")

    query = "INSERT INTO Employee VALUES (%s, %s, %s, %s)"
    cursor.execute(query, (emp_id, name, salary, designation))
    conn.commit()
    print("Employee Added")

def view_employees():
    cursor.execute("SELECT * FROM Employee")
    for row in cursor.fetchall():
        print(row)

def update_employee():
    emp_id = int(input("Enter ID to update: "))
    salary = int(input("Enter new Salary: "))

    query = "UPDATE Employee SET Salary=%s WHERE Emp_id=%s"
    cursor.execute(query, (salary, emp_id))
    conn.commit()
    print("Employee Updated")
```

```
def delete_employee():
    emp_id = int(input("Enter ID to delete: "))

    query = "DELETE FROM Employee WHERE Emp_id=%s"
    cursor.execute(query, (emp_id,))
    conn.commit()
    print("Employee Deleted")

while True:
    print("\n1.Add 2.View 3.Update 4.Delete 5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        add_employee()
    elif ch == 2:
        view_employees()
    elif ch == 3:
        update_employee()
    elif ch == 4:
        delete_employee()
    elif ch == 5:
        break

conn.close()
```

P2) Write an Unnamed PL/SQL of code for the following requirements: -
Schema:
Borrower (Rollin, Name, DateofIssue, NameofBook, Status)
Fine (Roll_no,Date,Amt)
Accept roll_no & name of book from user.
Check the number of days (from date of issue).
1. If days are between 15 to 30 then fine amounts will be Rs 5 per day.
2. If no. of days>30, per day fine will be Rs 50 per day & for days less than 30, Rs. 5 per day.
3. After submitting the book, status will change from I to R.
4. If condition of fine is true, then details will be stored into fine table.

Ans:

```
DECLARE
    v_roll      Borrower.Rollin%TYPE;
    v_book      Borrower.NameofBook%TYPE;
    v_name      Borrower.Name%TYPE;
```

```
    v_issue_dt  Borrower.DateofIssue%TYPE;
    v_days      NUMBER;
    v_fine      NUMBER := 0;
BEGIN
    v_roll := &Enter_Roll_No;
    v_book := '&Enter_Book_Name';
    SELECT Name, DateofIssue
    INTO v_name, v_issue_dt
    FROM Borrower
    WHERE Rollin = v_roll
    AND NameofBook = v_book
    AND Status = 'I';
    v_days := TRUNC(SYSDATE - v_issue_dt);
    IF v_days BETWEEN 15 AND 30 THEN
        v_fine := (v_days - 14) * 5;
    ELSIF v_days > 30 THEN
        v_fine := (16 * 5) + (v_days - 30) * 50;
    END IF;
    IF v_fine > 0 THEN
        INSERT INTO Fine (Roll_no, Date, Amt)
        VALUES (v_roll, SYSDATE, v_fine);
        DBMS_OUTPUT.PUT_LINE('Fine Amount: Rs ' || v_fine);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No Fine');
    END IF;

    -- Update status from I to R
    UPDATE Borrower
    SET Status = 'R'
    WHERE Rollin = v_roll
    AND NameofBook = v_book;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Record not found or already returned');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```